East West University

Department of Computer Science and Engineering (CSE)
Semester: (Spring, Year: 2026), B.Sc. in CSE

SOLID Principles Lab

Group: 2

Course Code: *CSE423*

Course Title: *Software Architecture*
Section: 01

Submitted to:

Ahmed Adnan
Lecturer, Department of Computer Science and Engineering

Students Details

Name	ID
Azahar Mahmud Chowdhury Rafi	2021-3-60-019
Sheikh Md Rakibul Hasan	2022-3-60-009
Ahnaf Anan	2023-1-60-089

Submission Date: 11th February, 2026

Refactored Application Layers:

- **Service Layer:** Addressing Dependency Inversion Principle (DIP) and Single Responsibility Principle (SRP) violations by introducing data access abstractions and separating validation logic.
- **CLI Layer:** Addressing Single Responsibility (SRP), Open/Closed Principle (OCP), and minor Dependency Inversion (DIP) violations in CliManager.
- **I/O Layer:** Addressing Open/Closed (OCP), Single Responsibility (SRP) in the IO layer.

SOLID Principles Implemented:

- **Dependency Inversion Principle (DIP):** Applied in the Service and CLI layer to depend on abstractions rather than concretions.
- **Single Responsibility Principle (SRP):** Applied across Service, CLI, I/O layers to ensure distinct class responsibilities.
- **Open/Closed Principle (OCP):** Implemented in CLI and I/O layers to enable extension without modification.

Git Repository Link [Initial Main Code]: https://github.com/Azahar-Mahmud/student-enrollment-system-refactored/commit/661e1dda4498167ad6d4f02c6208308e97385bb9

Service Layer Refactoring

1. Violation 1: Dependency Inversion Principle (DIP):

Files: StudentService.java, CourseService.java, ReportService.java, ImportExportService.java

Issue: Services depend on concrete DataStore class that exposes internal HashMap implementation

BEFORE: Services directly accessed HashMap through DataStore:
File: src\edu\ccrm\service\StudentService.java:

```java
package edu.ccrm.service;

    import java.util.Comparator;
    import java.util.List;
    import java.util.stream.Collectors;

    import edu.ccrm.domain.Course;
    import edu.ccrm.domain.Enrollment;
    import edu.ccrm.domain.Student;
    import edu.ccrm.exception.StudentNotFoundException;

    public class StudentService {
        private final DataStore dataStore;
        public StudentService(DataStore dataStore) {
            this.dataStore = dataStore;
        }

        public void addStudent(Student student) {
```

```java
        dataStore.getStudents().put(student.getRegNo(), student);
    }

    public Student findStudentByRegNo(String regNo) {
        Student student = dataStore.getStudents().get(regNo);
        if (student == null) {
            throw new StudentNotFoundException("Student with Registration No '" + regNo + "' not
found.");
        }
        return student;
    }

    public List<Student> getAllStudents() {
        return dataStore.getStudents().values().stream()
                .sorted(Comparator.comparing(Student::getRegNo))
                .collect(Collectors.toList());
    }

    public void updateStudentStatus(String regNo, Student.StudentStatus status) {
        Student student = findStudentByRegNo(regNo);
        student.setStatus(status);
    }

    public String getStudentTranscript(String regNo) {
        Student student = findStudentByRegNo(regNo);
        StringBuilder transcript = new StringBuilder();

        // Polymorphism: Calling getProfile() on a Student object
        transcript.append(student.getProfile()).append("\n\n");
        transcript.append("--- Academic Transcript ---\n");

        if (student.getEnrolledCourses().isEmpty()) {
            transcript.append("No courses enrolled yet.\n");
        } else {
            transcript.append(String.format("%-10s | %-30s | %-10s | %s\n", "Code", "Course
Title", "Credits", "Grade"));
            transcript.append("-----------------------------------------------------------------
\n");
            for (Enrollment enrollment : student.getEnrolledCourses()) {
                Course course = enrollment.getCourse();
                transcript.append(String.format("%-10s | %-30s | %-10d | %s\n",
                        course.getCourseCode(), course.getTitle(), course.getCredits(),
enrollment.getGrade()));
            }
        }
        transcript.append("-----------------------------------------------------------------\n");
        transcript.append(String.format("GPA: %.2f\n", calculateGpa(student)));

        return transcript.toString();
    }

    private double calculateGpa(Student student) {
        double totalPoints = 0;
        int totalCredits = 0;

        for (Enrollment en : student.getEnrolledCourses()) {
            if (en.getGrade().getGradePoint() >= 0) { // Exclude 'NA' grades
                totalPoints += en.getGrade().getGradePoint() * en.getCourse().getCredits();
                totalCredits += en.getCourse().getCredits();
            }
        }
        return totalCredits == 0 ? 0.0 : totalPoints / totalCredits;
    }
}
```

2. Violation 2: Single Responsibility Principle (SRP)

File: EnrollmentService.java

Issue: EnrollmentService contained an unnecessary inner class and had validation logic mixed with business logic.

BEFORE: validation logic was embedded directly in the enrollStudent method rather than being properly extracted.
File: src\edu\ccrm\service\ EnrollmentService.java:

```
package edu.ccrm.service;

import edu.ccrm.config.AppConfig;
import edu.ccrm.domain.Course;
import edu.ccrm.domain.Enrollment;
import edu.ccrm.domain.Grade;
import edu.ccrm.domain.Student;
import edu.ccrm.exception.DuplicateEnrollmentException;
import edu.ccrm.exception.MaxCreditLimitExceededException;

public class EnrollmentService {
    private final StudentService studentService;
    private final CourseService courseService;

    public EnrollmentService(StudentService ss, CourseService cs) {
        this.studentService = ss;
        this.courseService = cs;
    }

    /**
     * Enrolls a student in a course with business rule checks.
     * Demonstrates: throws clause for checked exceptions.
     */
    public void enrollStudent(String regNo, String courseCode)
            throws MaxCreditLimitExceededException, DuplicateEnrollmentException {
        Student student = studentService.findStudentByRegNo(regNo);
        Course course = courseService.findCourseByCode(courseCode);

        // Rule 1: Check for duplicate enrollment
        boolean alreadyEnrolled = student.getEnrolledCourses().stream()
                .anyMatch(e -> e.getCourse().getCourseCode().getFullCode().equals(courseCode));
        if (alreadyEnrolled) {
            throw new DuplicateEnrollmentException("Student " + regNo + " is already enrolled in
course " + courseCode);
        }

        // Rule 2: Check for max credit limit
        int currentCredits = student.getEnrolledCourses().stream()
                .filter(e -> e.getCourse().getSemester() == course.getSemester())
                .mapToInt(e -> e.getCourse().getCredits())
                .sum();

        int maxCredits = AppConfig.getInstance().getMaxCreditsPerSemester();
        if (currentCredits + course.getCredits() > maxCredits) {
            throw new MaxCreditLimitExceededException("Cannot enroll. Exceeds max credit limit of
" + maxCredits + " for the semester.");
        }

        Enrollment newEnrollment = new Enrollment(student, course);
        student.addEnrollment(newEnrollment);
        System.out.println("Enrollment successful!");
    }

    public void recordGrade(String regNo, String courseCode, int marks) {
        Student student = studentService.findStudentByRegNo(regNo);

        // Using an inner class to find the enrollment
        class EnrollmentFinder {
            Enrollment find() {
```

```
                for (Enrollment en : student.getEnrolledCourses()) {
                    if (en.getCourse().getCourseCode().getFullCode().equals(courseCode)) {
                        return en;
                    }
                }
                return null;
            }
        }

        EnrollmentFinder finder = new EnrollmentFinder();
        Enrollment enrollment = finder.find();

        if (enrollment != null) {
            enrollment.setGrade(Grade.fromMarks(marks));
            System.out.println("Grade recorded successfully for " + regNo + " in " + courseCode);
        } else {
            System.out.println("Error: Student " + regNo + " is not enrolled in course " +
courseCode);
        }
    }
}
```

AFTER: Applied Dependency Inversion Principle through abstraction interfaces.

1. Created Data Access Abstraction Layer
Created interfaces to decouple services from concrete storage implementation:

- IStudentRepository - Interface defining student data operations (abstraction)
- StudentRepositoryImpl - Concrete implementation using HashMap
- ICourseRepository - Interface defining course data operations (abstraction)
- CourseRepositoryImpl - Concrete implementation using HashMap
- IInstructorRepository - Interface defining instructor data operations (abstraction)
- InstructorRepositoryImpl - Concrete implementation using HashMap

This satisfies DIP: High-level services now depend on abstractions (interfaces), not concrete implementations (DataStore HashMap).
File: src/edu/ccrm/repository/IStudentRepository.java (NEW):

```
package edu.ccrm.repository;

import edu.ccrm.domain.Student;
import java.util.List;
import java.util.Optional;

public interface IStudentRepository {
    void save(Student student);
    Optional<Student> findByRegNo(String regNo);
    List<Student> findAll();
}
```

File: src/edu/ccrm/repository/ICourseRepository.java (NEW):

```
package edu.ccrm.repository;

import edu.ccrm.domain.Course;
import java.util.List;
import java.util.Optional;

public interface ICourseRepository {
    void save(Course course);
```

```
    Optional<Course> findByCode(String courseCode);
    List<Course> findAll();
}
```

File: src/edu/ccrm/repository/IInstructorRepository.java (NEW):

```
package edu.ccrm.repository;

import edu.ccrm.domain.Instructor;
import java.util.Optional;

public interface IInstructorRepository {
    void save(Instructor instructor);
    Optional<Instructor> findById(int id);
}
```

1. **Create Repository Implementations:**

File: src/edu/ccrm/repository/StudentRepositoryImpl.java (NEW):

```
package edu.ccrm.repository;

import edu.ccrm.domain.Student;
import java.util.*;

public class StudentRepositoryImpl implements IStudentRepository {
    private final Map<String, Student> storage = new HashMap<>();

    @Override
    public void save(Student student) {
        storage.put(student.getRegNo(), student);
    }

    @Override
    public Optional<Student> findByRegNo(String regNo) {
        return Optional.ofNullable(storage.get(regNo));
    }

    @Override
    public List<Student> findAll() {
        return new ArrayList<>(storage.values());
    }
}
```

File: src/edu/ccrm/repository/CourseRepositoryImpl.java (NEW):

```
package edu.ccrm.repository;

import edu.ccrm.domain.Course;
import java.util.*;

public class CourseRepositoryImpl implements ICourseRepository {
    private final Map<String, Course> storage = new HashMap<>();

    @Override
    public void save(Course course) {
        storage.put(course.getCourseCode().getFullCode(), course);
    }

    @Override
    public Optional<Course> findByCode(String code) {
        return Optional.ofNullable(storage.get(code));
```

```java
    }

    @Override
    public List<Course> findAll() {
        return new ArrayList<>(storage.values());
    }
}
```

File: src/edu/ccrm/repository/InstructorRepositoryImpl.java (NEW):

```java
package edu.ccrm.repository;

import edu.ccrm.domain.Instructor;
import java.util.*;

public class InstructorRepositoryImpl implements IInstructorRepository {
    private final Map<Integer, Instructor> storage = new HashMap<>();

    @Override
    public void save(Instructor instructor) {
        storage.put(instructor.getId(), instructor);
    }

    @Override
    public Optional<Instructor> findById(int id) {
        return Optional.ofNullable(storage.get(id));
    }
}
```

2. Refactor StudentService

File: src/edu/ccrm/service/StudentService.java (MODIFIED):

```java
package edu.ccrm.service;

import java.util.Comparator;
import java.util.List;
import java.util.stream.Collectors;

import edu.ccrm.domain.Course;
import edu.ccrm.domain.Enrollment;
import edu.ccrm.domain.Student;
import edu.ccrm.exception.StudentNotFoundException;
import edu.ccrm.repository.IStudentRepository;

public class StudentService {
    private final IStudentRepository studentRepository;

    public StudentService(IStudentRepository studentRepository) {
        this.studentRepository = studentRepository;
    }

    public void addStudent(Student student) {
        studentRepository.save(student);
    }

    public Student findStudentByRegNo(String regNo) {
        return studentRepository.findByRegNo(regNo)
            .orElseThrow(() -> new StudentNotFoundException(
                "Student with Registration No '" + regNo + "' not found."));
    }

    public List<Student> getAllStudents() {
        return studentRepository.findAll().stream()
```

```java
                .sorted(Comparator.comparing(Student::getRegNo))
                .collect(Collectors.toList());
    }

    public void updateStudentStatus(String regNo, Student.StudentStatus status) {
        Student student = findStudentByRegNo(regNo);
        student.setStatus(status);
        studentRepository.save(student);
    }
    public String getStudentTranscript(String regNo) {
        Student student = findStudentByRegNo(regNo);
        StringBuilder transcript = new StringBuilder();

        transcript.append(student.getProfile()).append("\n\n");
        transcript.append("--- Academic Transcript ---\n");

        if (student.getEnrolledCourses().isEmpty()) {
            transcript.append("No courses enrolled yet.\n");
        } else {
            transcript.append(String.format("%-10s | %-30s | %-10s | %s\n", "Code", "Course
Title", "Credits", "Grade"));
            transcript.append("-----------------------------------------------------------------
---\n");
            for (Enrollment enrollment : student.getEnrolledCourses()) {
                Course course = enrollment.getCourse();
                transcript.append(String.format("%-10s | %-30s | %-10d | %s\n",
                        course.getCourseCode(), course.getTitle(), course.getCredits(),
enrollment.getGrade()));
            }
        }
        transcript.append("-------------------------------------------------------------------
\n");
        transcript.append(String.format("GPA: %.2f\n", calculateGpa(student)));

        return transcript.toString();
    }

    private double calculateGpa(Student student) {
        double totalPoints = 0;
        int totalCredits = 0;

        for (Enrollment en : student.getEnrolledCourses()) {
            if (en.getGrade().getGradePoint() >= 0) {
                totalPoints += en.getGrade().getGradePoint() * en.getCourse().getCredits();
                totalCredits += en.getCourse().getCredits();
            }
        }
        return totalCredits == 0 ? 0.0 : totalPoints / totalCredits;
    }
}
```

3. Refactor EnrollmentService

File: src\edu\ccrm\service\EnrollmentService.java:

```java
package edu.ccrm.service;

import edu.ccrm.config.AppConfig;
import edu.ccrm.domain.Course;
import edu.ccrm.domain.Enrollment;
import edu.ccrm.domain.Grade;
import edu.ccrm.domain.Student;
import edu.ccrm.exception.DuplicateEnrollmentException;
import edu.ccrm.exception.MaxCreditLimitExceededException;

public class EnrollmentService {
    private final StudentService studentService;
```

```java
    private final CourseService courseService;

    public EnrollmentService(StudentService ss, CourseService cs) {
        this.studentService = ss;
        this.courseService = cs;
    }

    public void enrollStudent(String regNo, String courseCode)
            throws MaxCreditLimitExceededException, DuplicateEnrollmentException {
        Student student = studentService.findStudentByRegNo(regNo);
        Course course = courseService.findCourseByCode(courseCode);

        validateNoDuplicate(student, courseCode);
        validateCreditLimit(student, course);

        Enrollment newEnrollment = new Enrollment(student, course);
        student.addEnrollment(newEnrollment);
        System.out.println("Enrollment successful!");
    }

    public void recordGrade(String regNo, String courseCode, int marks) {
        Student student = studentService.findStudentByRegNo(regNo);

        Enrollment enrollment = findEnrollment(student, courseCode);

        if (enrollment != null) {
            enrollment.setGrade(Grade.fromMarks(marks));
            System.out.println("Grade recorded successfully for " + regNo + " in " +
courseCode);
        } else {
            System.out.println("Error: Student " + regNo + " is not enrolled in course " +
courseCode);
        }
    }

    private Enrollment findEnrollment(Student student, String courseCode) {
        return student.getEnrolledCourses().stream()
            .filter(en -> en.getCourse().getCourseCode().getFullCode().equals(courseCode))
            .findFirst()
            .orElse(null);
    }

    private void validateNoDuplicate(Student student, String courseCode)
            throws DuplicateEnrollmentException {
        boolean alreadyEnrolled = student.getEnrolledCourses().stream()
                .anyMatch(e ->
e.getCourse().getCourseCode().getFullCode().equals(courseCode));
        if (alreadyEnrolled) {
            throw new DuplicateEnrollmentException(
                "Student " + student.getRegNo() + " is already enrolled in course " +
courseCode);
        }
    }

    private void validateCreditLimit(Student student, Course course)
            throws MaxCreditLimitExceededException {
        int currentCredits = student.getEnrolledCourses().stream()
                .filter(e -> e.getCourse().getSemester() == course.getSemester())
                .mapToInt(e -> e.getCourse().getCredits())
                .sum();

        int maxCredits = AppConfig.getInstance().getMaxCreditsPerSemester();
        if (currentCredits + course.getCredits() > maxCredits) {
            throw new MaxCreditLimitExceededException(
                "Cannot enroll. Exceeds max credit limit of " + maxCredits + " for the
semester.");
        }
    }
}
```

SOLID Principles Applied:

- **DIP (Dependency Inversion Principle):** Services now depend on data access interfaces (abstractions) rather than concrete DataStore implementation. This allows easy swapping of storage mechanisms without changing service code.
- **SRP (Single Responsibility Principle):** Validation logic extracted into separate methods. Inner class removed and replaced with focused helper method. Each class/method now has single, well-defined responsibility

CLI Layer Refactoring

3. **Violation 3: Single Responsibility Principle (SRP) - God Class, Open/Closed Principle (OCP) & Dependency Inversion Principle (DIP):**
   File: CliManager.java
   Issue: One class handling 7+ responsibilities (God Class antipattern)
   BEFORE: CliManager doing everything and they have hardcoded switch statements

   File: src\edu\ccrm\cli\CliManager.java:

```java
package edu.ccrm.cli;

import java.time.LocalDate;
import java.util.Arrays;
import java.util.Scanner;
import java.util.function.Predicate;

import edu.ccrm.config.AppConfig;
import edu.ccrm.domain.Course;
import edu.ccrm.domain.Semester;
import edu.ccrm.domain.Student;
import edu.ccrm.exception.DuplicateEnrollmentException;
import edu.ccrm.exception.MaxCreditLimitExceededException;
import edu.ccrm.io.BackupService;
import edu.ccrm.io.ImportExportService;
import edu.ccrm.service.CourseService;
import edu.ccrm.service.DataStore;
import edu.ccrm.service.EnrollmentService;
import edu.ccrm.service.ReportService;
import edu.ccrm.service.StudentService;
import edu.ccrm.util.InputValidator;

/**
 * Main class for the Command-Line Interface.
 */
public class CliManager {

    private static final Scanner scanner = new Scanner(System.in);
    private static final DataStore dataStore = new DataStore();
    private static final StudentService studentService = new StudentService(dataStore);
    private static final CourseService courseService = new CourseService(dataStore);
    private static final EnrollmentService enrollmentService = new
EnrollmentService(studentService, courseService);
    private static final ReportService reportService = new ReportService(dataStore);
    private static final ImportExportService importExportService = new
ImportExportService(dataStore);
    private static final BackupService backupService = new BackupService();

    public static void main(String[] args) {
```

```java
        System.out.println("Welcome to the Student Course & Enrollment Management System!");
        System.out.println("Configuration loaded. Data directory: " +
AppConfig.getInstance().getDataDirectory());

        // Load initial data
        importExportService.importData();

        // Demonstrating a labeled jump with break
        OUTER_LOOP:
        while (true) {
            printMainMenu();
            int choice = InputValidator.getInt(scanner, "Enter your choice: ");
            switch (choice) {
                case 1 -> manageStudents();
                case 2 -> manageCourses();
                case 3 -> manageEnrollments();
                case 4 -> manageData();
                case 5 -> showReports();
                case 6 -> {
                    System.out.println("Exiting application...");
                    printPlatformNote();
                    break OUTER_LOOP; // Labeled break to exit the outer loop
                }
                default -> System.out.println("Invalid choice. Please try again.");
            }
        }
        scanner.close();
    }

    private static void printMainMenu() {
        System.out.println("\n--- Main Menu ---");
        System.out.println("1. Manage Students");
        System.out.println("2. Manage Courses");
        System.out.println("3. Manage Enrollments & Grades");
        System.out.println("4. Import/Export & Backup");
        System.out.println("5. View Reports");
        System.out.println("6. Exit");
    }

    private static void manageStudents() {
        System.out.println("\n--- Student Management ---");
        System.out.println("1. Add New Student");
        System.out.println("2. List All Students");
        System.out.println("3. View Student Profile & Transcript");
        System.out.println("4. Update Student Status");
        int choice = InputValidator.getInt(scanner, "Enter choice: ");

        switch (choice) {
            case 1 -> {
                System.out.println("Enter student details:");
                String regNo = InputValidator.getString(scanner, "Registration No: ");
                String name = InputValidator.getString(scanner, "Full Name: ");
                String email = InputValidator.getString(scanner, "Email: ");
                LocalDate dob = LocalDate.parse(InputValidator.getString(scanner, "Date of Birth
(YYYY-MM-DD): "));
                Student newStudent = new Student(dataStore.getStudents().size() + 1, regNo, name,
email, dob);
                studentService.addStudent(newStudent);
                System.out.println("Student added successfully.");
            }
            case 2 -> {
                System.out.println("\n--- List of All Students ---");
                studentService.getAllStudents().forEach(System.out::println);
            }
            case 3 -> {
                String regNo = InputValidator.getString(scanner, "Enter Student Registration No:
");
                try {
                    System.out.println(studentService.getStudentTranscript(regNo));
```

```
                } catch (Exception e) {
                    System.err.println("Error: " + e.getMessage());
                }
            }
            case 4 -> {
                 String regNo = InputValidator.getString(scanner, "Enter Student Registration No:
");
                 System.out.println("Available statuses: " +
Arrays.toString(Student.StudentStatus.values()));
                 String statusStr = InputValidator.getString(scanner, "Enter new status:
").toUpperCase();
                 try {
                     Student.StudentStatus status = Student.StudentStatus.valueOf(statusStr);
                     studentService.updateStudentStatus(regNo, status);
                     System.out.println("Status updated.");
                 } catch (IllegalArgumentException e) {
                     System.err.println("Invalid status provided.");
                 }
            }
            default -> System.out.println("Invalid choice.");
        }
    }

    private static void manageCourses() {
        System.out.println("\n--- Course Management ---");
        System.out.println("1. List All Courses");
        System.out.println("2. Search & Filter Courses");
        int choice = InputValidator.getInt(scanner, "Enter choice: ");

        switch (choice) {
            case 1 -> {
                System.out.println("\n--- List of All Courses ---");
                courseService.getAllCourses().forEach(System.out::println);
            }
            case 2 -> {
                System.out.println("Filter by: 1. Instructor, 2. Department, 3. Semester");
                int filterChoice = InputValidator.getInt(scanner, "Enter filter type: ");
                Predicate<Course> filter = course -> true; // Default: no filter

                if (filterChoice == 1) {
                    String name = InputValidator.getString(scanner, "Enter Instructor Name: ");
                    filter = CourseService.filterByInstructor(name);
                } else if (filterChoice == 2) {
                    String dept = InputValidator.getString(scanner, "Enter Department: ");
                    filter = CourseService.filterByDepartment(dept);
                } else if (filterChoice == 3) {
                    String semStr = InputValidator.getString(scanner, "Enter Semester (SPRING,
FALL, etc.): ");
                    try {
                        Semester sem = Semester.valueOf(semStr.toUpperCase());
                        filter = CourseService.filterBySemester(sem);
                    } catch (IllegalArgumentException e) {
                        System.err.println("Invalid semester.");
                        return;
                    }
                }
                System.out.println("\n--- Filtered Courses ---");
                courseService.searchCourses(filter).forEach(System.out::println);
            }
            default -> System.out.println("Invalid choice.");
        }
    }

    private static void manageEnrollments() {
        System.out.println("\n--- Enrollment & Grading ---");
        System.out.println("1. Enroll Student in Course");
        System.out.println("2. Record Grade for Student");
        int choice = InputValidator.getInt(scanner, "Enter choice: ");
```

```java
        String regNo = InputValidator.getString(scanner, "Enter Student Registration No: ");
        String courseCode = InputValidator.getString(scanner, "Enter Course Code (e.g., CS101): 
");

        switch (choice) {
            case 1 -> {
                try {
                    enrollmentService.enrollStudent(regNo, courseCode);
                } catch (MaxCreditLimitExceededException | DuplicateEnrollmentException e) {
                    System.err.println("Enrollment Failed: " + e.getMessage());
                } catch (RuntimeException e) { // Catches Student/Course not found
                    System.err.println("Error: " + e.getMessage());
                }
            }
            case 2 -> {
                int marks = InputValidator.getInt(scanner, "Enter marks (0-100): ");
                // Demonstrates try-with-resources (though scanner is static here, it's for syntax 
demo)
                // and multi-catch block.
                try (Scanner localScanner = new Scanner(System.in)) { // Just for demonstration
                     enrollmentService.recordGrade(regNo, courseCode, marks);
                } catch (NumberFormatException | NullPointerException e) {
                    System.err.println("An unexpected error occurred while processing grades: " + 
e.getMessage());
                } finally {
                    System.out.println("Grade recording process finished.");
                }
            }
            default -> System.out.println("Invalid choice.");
        }
    }

    private static void manageData() {
        System.out.println("\n--- Data Management ---");
        System.out.println("1. Export All Data");
        System.out.println("2. Create Backup of Exported Data");
        int choice = InputValidator.getInt(scanner, "Enter choice: ");

        switch (choice) {
            case 1 -> importExportService.exportData();
            case 2 -> backupService.createBackup();
            default -> System.out.println("Invalid choice.");
        }
    }

    private static void showReports() {
        System.out.println("\n--- Reports ---");
        System.out.println("1. GPA Distribution Report");
        int choice = InputValidator.getInt(scanner, "Enter choice: ");

        if (choice == 1) {
            reportService.generateGpaDistributionReport();
        } else {
            System.out.println("Invalid choice.");
        }
    }

    private static void printPlatformNote() {
        System.out.println("\n--- Java Platform Summary ---");
        System.out.println(" * Java SE (Standard Edition): Core Java platform for desktop and 
server applications.");
        System.out.println(" * Java EE (Enterprise Edition): Built on SE, adds APIs for large-
scale, distributed enterprise applications (e.g., web services, servlets).");
        System.out.println(" * Java ME (Micro Edition): A subset of SE for resource-constrained 
devices like mobile phones and embedded systems.");
        System.out.println("----------------------------\n");
    }
}
```

AFTER: Applied SRP, OCP, and DIP through operation separation.

1. **Create Command Abstraction & Implementations:**
   - MenuCommand.java - Operation interface defining the execute contract
   - ManageStudentsCommand - Handles all student management operations
   - ManageCoursesCommand - Handles course listing and filtering
   - ManageEnrollmentsCommand - Handles enrollment and grading
   - ManageDataCommand - Handles import/export/backup operations
   - ShowReportsCommand - Handles report generation

File: src/edu/ccrm/cli/command/MenuCommand.java (NEW)

```
package edu.ccrm.cli.command;

import java.util.Scanner;

public interface MenuCommand {
   void execute(Scanner scanner);
}
```

File: src/edu/ccrm/cli/command/ManageStudentsCommand.java (NEW):

```
package edu.ccrm.cli.command;

import java.time.LocalDate;
import java.util.Arrays;
import java.util.Scanner;
import edu.ccrm.domain.Student;
import edu.ccrm.repository.IStudentRepository;
import edu.ccrm.service.StudentService;
import edu.ccrm.util.InputValidator;

public class ManageStudentsCommand implements MenuCommand {
    private final StudentService studentService;
    private final IStudentRepository studentRepo;

    public ManageStudentsCommand(StudentService studentService, IStudentRepository
studentRepo) {
        this.studentService = studentService;
        this.studentRepo = studentRepo;
    }

    @Override
    public void execute(Scanner scanner) {
        System.out.println("\n--- Student Management ---");
        System.out.println("1. Add New Student");
        System.out.println("2. List All Students");
        System.out.println("3. View Student Profile & Transcript");
        System.out.println("4. Update Student Status");

        int choice = InputValidator.getInt(scanner, "Enter choice: ");
        switch (choice) {
            case 1 -> addStudent(scanner);
            case 2 -> listStudents();
            case 3 -> viewTranscript(scanner);
            case 4 -> updateStatus(scanner);
            default -> System.out.println("Invalid choice.");
        }
    }

    private void addStudent(Scanner scanner) {
        System.out.println("Enter student details:");
        String regNo = InputValidator.getString(scanner, "Registration No: ");
        String name = InputValidator.getString(scanner, "Full Name: ");
```

```
        String email = InputValidator.getString(scanner, "Email: ");
        LocalDate dob = LocalDate.parse(InputValidator.getString(scanner, "Date of Birth
(YYYY-MM-DD): "));

        int newId = studentRepo.findAll().size() + 1;
        Student newStudent = new Student(newId, regNo, name, email, dob);
        studentService.addStudent(newStudent);
        System.out.println("Student added successfully.");
    }

    private void listStudents() {
        System.out.println("\n--- List of All Students ---");
        studentService.getAllStudents().forEach(System.out::println);
    }

    private void viewTranscript(Scanner scanner) {
        String regNo = InputValidator.getString(scanner, "Enter Student Registration No: ");
        try {
            System.out.println(studentService.getStudentTranscript(regNo));
        } catch (Exception e) {
            System.err.println("Error: " + e.getMessage());
        }
    }

    private void updateStatus(Scanner scanner) {
        String regNo = InputValidator.getString(scanner, "Enter Student Registration No: ");
        System.out.println("Available statuses: " +
Arrays.toString(Student.StudentStatus.values()));
        String statusStr = InputValidator.getString(scanner, "Enter new status:
").toUpperCase();
        try {
            Student.StudentStatus status = Student.StudentStatus.valueOf(statusStr);
            studentService.updateStudentStatus(regNo, status);
            System.out.println("Status updated.");
        } catch (IllegalArgumentException e) {
            System.err.println("Invalid status provided.");
        }
    }
}
```

File: src/edu/ccrm/cli/command/ ManageCoursesCommand.java (NEW):

```
package edu.ccrm.cli.command;

import java.util.Scanner;
import java.util.function.Predicate;
import edu.ccrm.domain.Course;
import edu.ccrm.domain.Semester;
import edu.ccrm.service.CourseService;
import edu.ccrm.util.InputValidator;

public class ManageCoursesCommand implements MenuCommand {
    private final CourseService courseService;

    public ManageCoursesCommand(CourseService courseService) {
        this.courseService = courseService;
    }

    @Override
    public void execute(Scanner scanner) {
        System.out.println("\n--- Course Management ---");
        System.out.println("1. List All Courses");
        System.out.println("2. Search & Filter Courses");
```

```java
        int choice = InputValidator.getInt(scanner, "Enter choice: ");
        switch (choice) {
            case 1 -> listCourses();
            case 2 -> filterCourses(scanner);
            default -> System.out.println("Invalid choice.");
        }
    }

    private void listCourses() {
        System.out.println("\n--- List of All Courses ---");
        courseService.getAllCourses().forEach(System.out::println);
    }

    private void filterCourses(Scanner scanner) {
        System.out.println("Filter by: 1. Instructor, 2. Department, 3.
Semester");
        int filterChoice = InputValidator.getInt(scanner, "Enter filter
type: ");
        Predicate<Course> filter = course -> true;

        if (filterChoice == 1) {
            String name = InputValidator.getString(scanner, "Enter
Instructor Name: ");
            filter = CourseService.filterByInstructor(name);
        } else if (filterChoice == 2) {
            String dept = InputValidator.getString(scanner, "Enter
Department: ");
            filter = CourseService.filterByDepartment(dept);
        } else if (filterChoice == 3) {
            String semStr = InputValidator.getString(scanner, "Enter
Semester (SPRING, FALL, etc.): ");
            try {
                Semester sem = Semester.valueOf(semStr.toUpperCase());
                filter = CourseService.filterBySemester(sem);
            } catch (IllegalArgumentException e) {
                System.err.println("Invalid semester.");
                return;
            }
        }
        System.out.println("\n--- Filtered Courses ---");
        courseService.searchCourses(filter).forEach(System.out::println);
    }
}
```

File: src/edu/ccrm/cli/command/ ManageEnrollmentsCommand.java (NEW):

```java
package edu.ccrm.cli.command;

import java.util.Scanner;
import edu.ccrm.exception.DuplicateEnrollmentException;
import edu.ccrm.exception.MaxCreditLimitExceededException;
import edu.ccrm.service.EnrollmentService;
import edu.ccrm.util.InputValidator;
```

```java
public class ManageEnrollmentsCommand implements MenuCommand {
    private final EnrollmentService enrollmentService;

    public ManageEnrollmentsCommand(EnrollmentService enrollmentService) {
        this.enrollmentService = enrollmentService;
    }

    @Override
    public void execute(Scanner scanner) {
        System.out.println("\n--- Enrollment & Grading ---");
        System.out.println("1. Enroll Student in Course");
        System.out.println("2. Record Grade for Student");

        int choice = InputValidator.getInt(scanner, "Enter choice: ");
        String regNo = InputValidator.getString(scanner, "Enter Student
Registration No: ");
        String courseCode = InputValidator.getString(scanner, "Enter Course
Code (e.g., CS101): ");

        switch (choice) {
            case 1 -> enrollStudent(regNo, courseCode);
            case 2 -> recordGrade(scanner, regNo, courseCode);
            default -> System.out.println("Invalid choice.");
        }
    }

    private void enrollStudent(String regNo, String courseCode) {
        try {
            enrollmentService.enrollStudent(regNo, courseCode);
        } catch (MaxCreditLimitExceededException |
DuplicateEnrollmentException e) {
            System.err.println("Enrollment Failed: " + e.getMessage());
        } catch (RuntimeException e) {
            System.err.println("Error: " + e.getMessage());
        }
    }

    private void recordGrade(Scanner scanner, String regNo, String
courseCode) {
        int marks = InputValidator.getInt(scanner, "Enter marks (0-100):
");
        try {
            enrollmentService.recordGrade(regNo, courseCode, marks);
        } catch (NumberFormatException | NullPointerException e) {
            System.err.println("An unexpected error occurred while
processing grades: " + e.getMessage());
        }
    }
}
```

File: src/edu/ccrm/cli/command/ ManageDataCommand.java (NEW):

```java
package edu.ccrm.cli.command;
```

```java
import java.util.Scanner;
import edu.ccrm.io.BackupService;
import edu.ccrm.io.ImportExportService;
import edu.ccrm.util.InputValidator;

public class ManageDataCommand implements MenuCommand {
    private final ImportExportService importExportService;
    private final BackupService backupService;

    public ManageDataCommand(ImportExportService importExportService,
BackupService backupService) {
        this.importExportService = importExportService;
        this.backupService = backupService;
    }

    @Override
    public void execute(Scanner scanner) {
        System.out.println("\n--- Data Management ---");
        System.out.println("1. Export All Data");
        System.out.println("2. Create Backup of Exported Data");

        int choice = InputValidator.getInt(scanner, "Enter choice: ");
        switch (choice) {
            case 1 -> importExportService.exportData();
            case 2 -> backupService.createBackup();
            default -> System.out.println("Invalid choice.");
        }
    }
}
```

File: src/edu/ccrm/cli/command/ ShowReportsCommand.java (NEW):

```java
package edu.ccrm.cli.command;

import java.util.Scanner;
import edu.ccrm.service.ReportService;
import edu.ccrm.util.InputValidator;

public class ShowReportsCommand implements MenuCommand {
    private final ReportService reportService;

    public ShowReportsCommand(ReportService reportService) {
        this.reportService = reportService;
    }

    @Override
    public void execute(Scanner scanner) {
        System.out.println("\n--- Reports ---");
        System.out.println("1. GPA Distribution Report");

        int choice = InputValidator.getInt(scanner, "Enter choice: ");
        if (choice == 1) {
            reportService.generateGpaDistributionReport();
```

```
        } else {
            System.out.println("Invalid choice.");
        }
    }
}
```

2. **Refactor CliManager**
   File: src/edu/ccrm/cli/CliManager.java (MODIFIED):

```
package edu.ccrm.cli;

import java.util.HashMap;
import java.util.Map;
import java.util.Scanner;

import edu.ccrm.cli.command.ManageCoursesCommand;
import edu.ccrm.cli.command.ManageDataCommand;
import edu.ccrm.cli.command.ManageEnrollmentsCommand;
import edu.ccrm.cli.command.ManageStudentsCommand;
import edu.ccrm.cli.command.MenuCommand;
import edu.ccrm.cli.command.ShowReportsCommand;
import edu.ccrm.config.AppConfig;
import edu.ccrm.io.BackupService;
import edu.ccrm.io.ImportExportService;
import edu.ccrm.repository.CourseRepositoryImpl;
import edu.ccrm.repository.ICourseRepository;
import edu.ccrm.repository.IInstructorRepository;
import edu.ccrm.repository.IStudentRepository;
import edu.ccrm.repository.InstructorRepositoryImpl;
import edu.ccrm.repository.StudentRepositoryImpl;
import edu.ccrm.service.CourseService;
import edu.ccrm.service.EnrollmentService;
import edu.ccrm.service.ReportService;
import edu.ccrm.service.StudentService;
import edu.ccrm.util.InputValidator;

public class CliManager {
    private static final Scanner scanner = new Scanner(System.in);

    private static final IStudentRepository studentRepo = new StudentRepositoryImpl();
    private static final ICourseRepository courseRepo = new CourseRepositoryImpl();
    private static final IInstructorRepository instructorRepo = new
InstructorRepositoryImpl();

    private static final StudentService studentService = new StudentService(studentRepo);
    private static final CourseService courseService = new CourseService(courseRepo);
    private static final EnrollmentService enrollmentService = new
EnrollmentService(studentService, courseService);
    private static final ReportService reportService = new ReportService(studentRepo);
    private static final ImportExportService importExportService = new
ImportExportService(studentRepo, courseRepo, instructorRepo);
    private static final BackupService backupService = new BackupService();

    private static final Map<Integer, MenuCommand> commands = new HashMap<>();

    static {
        commands.put(1, new ManageStudentsCommand(studentService, studentRepo));
        commands.put(2, new ManageCoursesCommand(courseService));
        commands.put(3, new ManageEnrollmentsCommand(enrollmentService));
        commands.put(4, new ManageDataCommand(importExportService, backupService));
        commands.put(5, new ShowReportsCommand(reportService));
```

```java
    }

    public static void main(String[] args) {
        System.out.println("Welcome to the Student Course & Enrollment Management System!");
        System.out.println("Configuration loaded. Data directory: " +
AppConfig.getInstance().getDataDirectory());

        importExportService.importData();

        while (true) {
            printMainMenu();
            int choice = InputValidator.getInt(scanner, "Enter your choice: ");

            if (choice == 6) {
                System.out.println("Exiting application...");
                printPlatformNote();
                break;
            }

            MenuCommand command = commands.get(choice);
            if (command != null) {
                command.execute(scanner);
            } else {
                System.out.println("Invalid choice. Please try again.");
            }
        }
        scanner.close();
    }

    private static void printMainMenu() {
        System.out.println("\n--- Main Menu ---");
        System.out.println("1. Manage Students");
        System.out.println("2. Manage Courses");
        System.out.println("3. Manage Enrollments & Grades");
        System.out.println("4. Import/Export & Backup");
        System.out.println("5. View Reports");
        System.out.println("6. Exit");
    }

    private static void printPlatformNote() {
        System.out.println("\n--- Java Platform Summary ---");
        System.out.println(" * Java SE (Standard Edition): Core Java platform for desktop and
server applications.");
        System.out.println(" * Java EE (Enterprise Edition): Built on SE, adds APIs for large-
scale, distributed enterprise applications.");
        System.out.println(" * Java ME (Micro Edition): A subset of SE for resource-
constrained devices.");
        System.out.println("----------------------------\n");
    }
}
```

SOLID Principles Applied:

- **SRP (Single Responsibility Principle):** CliManager's 7+ responsibilities extracted into separate operation classes. Each class handles only one menu operation (student management, course management, etc.).
- **OCP (Open/Closed Principle):** New menu operations can be added by creating new operation classes without modifying CliManager or existing operations. The switch-case is replaced with a map-based dispatcher that's open for extension but closed for modification.
- **DIP (Dependency Inversion Principle):** CliManager depends on MenuCommand interface (abstraction) rather than concrete operation classes. Operations are injected through the map, allowing flexible addition/removal.

IO Layer Refactoring

3. Violation 4: Single Responsibility Principle (SRP) & Open/Closed Principle (OCP):

File: ImportExportService.java

Issue:

SRP Violation: ImportExportService handled 6+ different responsibilities: Import instructors, courses, students, Export students, courses, enrollments. Each responsibility should be in its own class.

OCP Violation: Must modify ImportExportService class directly to add new CSV export/import types. Cannot extend without modification.

BEFORE: ImportExportService handled 6+ different responsibilities:

- Import instructors
- Import courses
- Import students
- Export students
- Export courses
- Export enrollments

File: src\edu\ccrm\io\ImportExportService.java:

```java
package edu.ccrm.io;

import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.time.LocalDate;
import java.util.List;
import java.util.stream.Collectors;
import java.util.stream.Stream;

import edu.ccrm.config.AppConfig;
import edu.ccrm.domain.Course;
import edu.ccrm.domain.CourseCode;
import edu.ccrm.domain.Instructor;
import edu.ccrm.domain.Semester;
import edu.ccrm.domain.Student;
import edu.ccrm.service.DataStore;

/**
 * Service for importing and exporting data using NIO.2 and Streams.
 */
public class ImportExportService {
    private final DataStore dataStore;
    private final Path dataDir = Paths.get(AppConfig.getInstance().getDataDirectory());

    public ImportExportService(DataStore dataStore) {
        this.dataStore = dataStore;
    }

    public void importData() {
        try {
            if (!Files.exists(dataDir)) {
```

```
                Files.createDirectories(dataDir);
            }
            importInstructors();
            importCourses();
            importStudents();
            System.out.println("Data imported successfully from '" + dataDir + "' directory.");
        } catch (IOException e) {
            System.err.println("Error during data import: " + e.getMessage());
        }
    }

    // In a real app, instructors would also be in a CSV
    private void importInstructors() {
        dataStore.getInstructors().put(1, new Instructor(1, "Dr. Alan Turing",
"alan.t@bletchley.uk", LocalDate.of(1912, 6, 23), "Computer Science", "A101"));
        dataStore.getInstructors().put(2, new Instructor(2, "Dr. Grace Hopper",
"grace.h@yale.edu", LocalDate.of(1906, 12, 9), "Computer Science", "B202"));
    }
    private void importCourses() throws IOException {
        Path coursesFile = dataDir.resolve("courses.csv");
        if (!Files.exists(coursesFile)) return;

        try (Stream<String> lines = Files.lines(coursesFile)) {
            lines.skip(1) // Skip header
                 .map(line -> line.split(","))
                 .forEach(parts -> {
                     CourseCode code = new CourseCode(parts[0].replaceAll("\\d", ""),
Integer.parseInt(parts[0].replaceAll("\\D", "")));
                     Instructor instructor =
dataStore.getInstructors().get(Integer.parseInt(parts[4]));
                     Course course = new Course.Builder(code, parts[1])
                                             .credits(Integer.parseInt(parts[2]))
                                             .semester(Semester.valueOf(parts[3].toUpperCase()))
                                             .instructor(instructor)
                                             .department(parts[5])
                                             .build();
                     dataStore.getCourses().put(code.getFullCode(), course);
                 });
        }
    }

    private void importStudents() throws IOException {
        Path studentsFile = dataDir.resolve("students.csv");
        if (!Files.exists(studentsFile)) return;

        try (Stream<String> lines = Files.lines(studentsFile)) {
            lines.skip(1) // Skip header
                 .map(line -> line.split(","))
                 .forEach(parts -> {
                    Student student = new Student(Integer.parseInt(parts[0]), parts[1], parts[2],
parts[3], LocalDate.parse(parts[4]));
                    dataStore.getStudents().put(student.getRegNo(), student);
                 });
        }
    }

    public void exportData() {
        try {
            if (!Files.exists(dataDir)) {
                Files.createDirectories(dataDir);
            }
            exportStudents();
            exportCourses();
            exportEnrollments();
            System.out.println("Data exported successfully to '" + dataDir + "' directory.");
        } catch (IOException e) {
            System.err.println("Error during data export: " + e.getMessage());
        }
    }
```

```java
    private void exportStudents() throws IOException {
        Path studentsFile = dataDir.resolve("students_export.csv");
        List<String> lines = dataStore.getStudents().values().stream()
                .map(s -> String.join(",", String.valueOf(s.getId()), s.getRegNo(),
s.getFullName(), s.getEmail(), s.getDateOfBirth().toString(), s.getStatus().name()))
                .collect(Collectors.toList());
        lines.add(0, "id,regNo,fullName,email,dob,status");
        Files.write(studentsFile, lines);
    }

    private void exportCourses() throws IOException {
         Path coursesFile = dataDir.resolve("courses_export.csv");
        List<String> lines = dataStore.getCourses().values().stream()
                .map(c -> String.join(",", c.getCourseCode().getFullCode(), c.getTitle(),
String.valueOf(c.getCredits()), c.getSemester().name(), String.valueOf(c.getInstructor().getId()),
c.getDepartment()))
                .collect(Collectors.toList());
        lines.add(0, "courseCode,title,credits,semester,instructorId,department");
        Files.write(coursesFile, lines);
    }

    private void exportEnrollments() throws IOException {
        Path enrollmentsFile = dataDir.resolve("enrollments_export.csv");
        List<String> lines = dataStore.getStudents().values().stream()
                .flatMap(s -> s.getEnrolledCourses().stream())
                .map(en -> String.join(",", en.getStudent().getRegNo(),
en.getCourse().getCourseCode().getFullCode(), en.getGrade().name()))
                .collect(Collectors.toList());
        lines.add(0, "regNo,courseCode,grade");
        Files.write(enrollmentsFile, lines);
    }
}
```

AFTER: Applied OCP and SRP through abstract base classes and specific handlers

1. **Create Export and Import Interfaces**
   File: src/edu/ccrm/io/strategy/ICsvExporter.java (NEW):

```java
package edu.ccrm.io.strategy;

import java.io.IOException;

public interface ICsvExporter {
    void export() throws IOException;
}
```

2. **Create Abstract Import Handler**
   File: src/edu/ccrm/io/strategy/ICsvImporter.java (NEW):

```java
package edu.ccrm.io.strategy;

import java.io.IOException;

public interface ICsvImporter {
    void importData() throws IOException;
}
```

3. **Create Concrete Handlers**
   File: src/edu/ccrm/io/strategy/StudentCsvExporter.java (NEW):

```java
package edu.ccrm.io.strategy;

import java.io.IOException;
```

```java
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.List;
import java.util.stream.Collectors;

import edu.ccrm.repository.IStudentRepository;

public class StudentCsvExporter implements ICsvExporter {
    private final IStudentRepository studentRepo;
    private final Path dataDir;

    public StudentCsvExporter(IStudentRepository studentRepo, Path dataDir) {
        this.studentRepo = studentRepo;
        this.dataDir = dataDir;
    }

    @Override
    public void export() throws IOException {
        if (!Files.exists(dataDir)) {
            Files.createDirectories(dataDir);
        }

        Path filePath = dataDir.resolve("students_export.csv");

        List<String> lines = studentRepo.findAll().stream()
            .map(s -> String.format("%d,%s,%s,%s,%s,%s",
                s.getId(), s.getRegNo(), s.getFullName(),
                s.getEmail(), s.getDateOfBirth(), s.getStatus()))
            .collect(Collectors.toList());

        lines.add(0, "id,regNo,fullName,email,dob,status");
        Files.write(filePath, lines);
    }
}
```

File: src/edu/ccrm/io/strategy/CourseCsvExporter.java (NEW):

```java
package edu.ccrm.io.strategy;

import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.List;
import java.util.stream.Collectors;

import edu.ccrm.repository.ICourseRepository;

public class CourseCsvExporter implements ICsvExporter {
    private final ICourseRepository courseRepo;
    private final Path dataDir;

    public CourseCsvExporter(ICourseRepository courseRepo, Path dataDir) {
        this.courseRepo = courseRepo;
        this.dataDir = dataDir;
    }

    @Override
    public void export() throws IOException {
        if (!Files.exists(dataDir)) {
            Files.createDirectories(dataDir);
        }

        Path filePath = dataDir.resolve("courses_export.csv");

        List<String> lines = courseRepo.findAll().stream()
            .map(c -> String.format("%s,%s,%d,%s,%d,%s",
                c.getCourseCode().getFullCode(), c.getTitle(),
```

```
                c.getCredits(), c.getSemester(),
                c.getInstructor().getId(), c.getDepartment()))
            .collect(Collectors.toList());

        lines.add(0, "courseCode,title,credits,semester,instructorId,department");
        Files.write(filePath, lines);
    }
}
```

File: src/edu/ccrm/io/strategy/EnrollmentCsvExporter.java (NEW):

```
package edu.ccrm.io.strategy;

import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.List;
import java.util.stream.Collectors;

import edu.ccrm.repository.IStudentRepository;

public class EnrollmentCsvExporter implements ICsvExporter {
    private final IStudentRepository studentRepo;
    private final Path dataDir;

    public EnrollmentCsvExporter(IStudentRepository studentRepo, Path dataDir) {
        this.studentRepo = studentRepo;
        this.dataDir = dataDir;
    }

    @Override
    public void export() throws IOException {
        if (!Files.exists(dataDir)) {
            Files.createDirectories(dataDir);
        }

        Path filePath = dataDir.resolve("enrollments_export.csv");

        List<String> lines = studentRepo.findAll().stream()
            .flatMap(s -> s.getEnrolledCourses().stream()
                .map(en -> String.format("%s,%s,%s",
                    s.getRegNo(),
                    en.getCourse().getCourseCode().getFullCode(),
                    en.getGrade())))
            .collect(Collectors.toList());

        lines.add(0, "regNo,courseCode,grade");
        Files.write(filePath, lines);
    }
}
```

File: src/edu/ccrm/io/strategy/StudentCsvImporter.java (NEW):

```
package edu.ccrm.io.strategy;

import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.time.LocalDate;
import java.util.stream.Stream;

import edu.ccrm.domain.Student;
import edu.ccrm.repository.IStudentRepository;

public class StudentCsvImporter implements ICsvImporter {
    private final IStudentRepository studentRepo;
    private final Path dataDir;
```

```java
    public StudentCsvImporter(IStudentRepository studentRepo, Path dataDir) {
        this.studentRepo = studentRepo;
        this.dataDir = dataDir;
    }

    @Override
    public void importData() throws IOException {
        Path filePath = dataDir.resolve("students.csv");

        if (!Files.exists(filePath)) {
            return;
        }

        try (Stream<String> lines = Files.lines(filePath)) {
            lines.skip(1)
                .map(line -> line.split(","))
                .forEach(parts -> {
                    Student student = new Student(
                        Integer.parseInt(parts[0]),
                        parts[1],
                        parts[2],
                        parts[3],
                        LocalDate.parse(parts[4])
                    );
                    studentRepo.save(student);
                });
        }
    }
}
```

File: src/edu/ccrm/io/strategy/CourseCsvImporter.java (NEW):

```java
package edu.ccrm.io.strategy;

import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.stream.Stream;

import edu.ccrm.domain.Course;
import edu.ccrm.domain.CourseCode;
import edu.ccrm.domain.Instructor;
import edu.ccrm.domain.Semester;
import edu.ccrm.repository.ICourseRepository;
import edu.ccrm.repository.IInstructorRepository;

public class CourseCsvImporter implements ICsvImporter {
    private final ICourseRepository courseRepo;
    private final IInstructorRepository instructorRepo;
    private final Path dataDir;

    public CourseCsvImporter(ICourseRepository courseRepo,
                             IInstructorRepository instructorRepo,
                             Path dataDir) {
        this.courseRepo = courseRepo;
        this.instructorRepo = instructorRepo;
        this.dataDir = dataDir;
    }

    @Override
    public void importData() throws IOException {
        Path filePath = dataDir.resolve("courses.csv");

        if (!Files.exists(filePath)) {
            return;
        }
```

```java
        try (Stream<String> lines = Files.lines(filePath)) {
            lines.skip(1)
                .map(line -> line.split(","))
                .forEach(parts -> {
                    String deptPrefix = parts[0].replaceAll("\\d", "");
                    int courseNum = Integer.parseInt(parts[0].replaceAll("\\D", ""));
                    CourseCode code = new CourseCode(deptPrefix, courseNum);

                    Instructor instructor = instructorRepo
                        .findById(Integer.parseInt(parts[4]))
                        .orElseThrow(() -> new RuntimeException(
                            "Instructor not found"));

                    Course course = new Course.Builder(code, parts[1])
                        .credits(Integer.parseInt(parts[2]))
                        .semester(Semester.valueOf(parts[3].toUpperCase()))
                        .instructor(instructor)
                        .department(parts[5])
                        .build();

                    courseRepo.save(course);
                });
        }
    }
}
```

4. Refactor ImportExportService

File: src/edu/ccrm/io/ImportExportService.java (MODIFIED):

```java
package edu.ccrm.io;

import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.time.LocalDate;
import java.util.Arrays;
import java.util.List;

import edu.ccrm.config.AppConfig;
import edu.ccrm.domain.Instructor;
import edu.ccrm.io.strategy.CourseCsvExporter;
import edu.ccrm.io.strategy.CourseCsvImporter;
import edu.ccrm.io.strategy.EnrollmentCsvExporter;
import edu.ccrm.io.strategy.ICsvExporter;
import edu.ccrm.io.strategy.ICsvImporter;
import edu.ccrm.io.strategy.StudentCsvExporter;
import edu.ccrm.io.strategy.StudentCsvImporter;
import edu.ccrm.repository.ICourseRepository;
import edu.ccrm.repository.IInstructorRepository;
import edu.ccrm.repository.IStudentRepository;

public class ImportExportService {
    private final IInstructorRepository instructorRepo;
    private final List<ICsvImporter> importers;
    private final List<ICsvExporter> exporters;
    private final Path dataDir;

    public ImportExportService(IStudentRepository studentRepo,
                               ICourseRepository courseRepo,
                               IInstructorRepository instructorRepo) {
        this.instructorRepo = instructorRepo;
        this.dataDir = Paths.get(AppConfig.getInstance().getDataDirectory());

        this.importers = Arrays.asList(
            new StudentCsvImporter(studentRepo, dataDir),
```

```java
            new CourseCsvImporter(courseRepo, instructorRepo, dataDir)
        );

        this.exporters = Arrays.asList(
            new StudentCsvExporter(studentRepo, dataDir),
            new CourseCsvExporter(courseRepo, dataDir),
            new EnrollmentCsvExporter(studentRepo, dataDir)
        );
    }

    public void importData() {
        try {
            if (!Files.exists(dataDir)) {
                Files.createDirectories(dataDir);
            }

            importInstructors();

            for (ICsvImporter importer : importers) {
                importer.importData();
            }

            System.out.println("Data imported successfully from '"
                + dataDir + "' directory.");
        } catch (IOException e) {
            System.err.println("Error during data import: " + e.getMessage());
        }
    }

    public void exportData() {
        try {
            if (!Files.exists(dataDir)) {
                Files.createDirectories(dataDir);
            }

            for (ICsvExporter exporter : exporters) {
                exporter.export();
            }

            System.out.println("Data exported successfully to '"
                + dataDir + "' directory.");
        } catch (IOException e) {
            System.err.println("Error during data export: " + e.getMessage());
        }
    }

    private void importInstructors() {
        instructorRepo.save(new Instructor(1, "Dr. Alan Turing",
            "alan.t@bletchley.uk", LocalDate.of(1912, 6, 23),
            "Computer Science", "A101"));
        instructorRepo.save(new Instructor(2, "Dr. Grace Hopper",
            "grace.h@yale.edu", LocalDate.of(1906, 12, 9),
            "Computer Science", "B202"));
    }
}
```

SOLID Principles Applied:

- **OCP (Open/Closed Principle):** Interface abstractions (ICsvExporter, ICsvImporter) define contracts for CSV operations. New CSV types are added by creating new concrete handler classes that implement these interfaces. ImportExportService doesn't need modification when adding new CSV types. System is open for extension (new handlers) but closed for modification (existing code unchanged).

- **SRP (Single Responsibility Principle):** Each concrete handler (StudentCsvExporter, CourseCsvImporter, etc.) handles exactly ONE entity type. ImportExportService only coordinates handlers, doesn't contain import/export logic. Separation of concerns: interfaces define contracts, concrete classes implement specifics.